

Parte III - Ejercicios aplicando pandas

Alumna: Victoria Maria Chavarría Villeda

Ejercicio 16: DataFrame de estudiantes

- Cree un DataFrame con al menos 10 estudiantes.
- Incluya nombre, carrera, edad, promedio y ciudad.
- Muestre las primeras 5 filas.
- Muestre la información general del DataFrame.
- Calcule el promedio general de calificaciones.
- Muestre estudiantes con promedio mayor o igual a 80.
- Ordene estudiantes por promedio de mayor a menor.

```
import pandas as pd
```

```
# Crear DataFrame
```

```
estudiantes = {  
    "Nombre": ["Ana", "Luis", "Carlos", "María", "Pedro",  
               "Sofía", "José", "Elena", "Miguel", "Laura"],  
    "Carrera": ["Sistemas", "Medicina", "Derecho", "Sistemas", "Contaduría",  
                "Medicina", "Arquitectura", "Sistemas", "Derecho", "Contaduría"],  
    "Edad": [20, 22, 21, 19, 23, 20, 24, 22, 21, 20],  
    "Promedio": [85, 78, 90, 88, 70, 95, 82, 79, 84, 91],  
    "Ciudad": ["Tegucigalpa", "San Pedro", "La Ceiba", "Comayagua", "Choluteca",  
               "Tegucigalpa", "Danlí", "Siguatepeque", "La Paz", "Comayagua"]  
}
```

```
df = pd.DataFrame(estudiantes)
```

```
# Primeras 5 filas
```

```
print(df.head())
```

```
# Información general
```

```
print(df.info())
```

```
# Promedio general
```

```
print("Promedio general:", df["Promedio"].mean())
```

```
# Estudiantes con promedio >=80
```

```
print(df[df["Promedio"] >= 80])
```

```
# Ordenar de mayor a menor
```

```
print(df.sort_values(by="Promedio", ascending=False))
```

Ejercicio 17: Análisis de ventas con pandas

- Cree un DataFrame de ventas con al menos 15 registros.
- Incluya fecha, vendedor, producto, categoría, cantidad y precio unitario.
- Calcule una nueva columna TotalVenta.
- Calcule el total vendido por vendedor.
- Identifique el producto más vendido.
- Identifique la categoría con mayor ingreso.
- Ordene las ventas de mayor a menor.

```
import pandas as pd
```

```
ventas = {
```

```
    "Fecha": ["2026-01-01", "2026-01-02", "2026-01-03", "2026-01-04", "2026-01-05",  
              "2026-01-06", "2026-01-07", "2026-01-08", "2026-01-09", "2026-01-10",  
              "2026-01-11", "2026-01-12", "2026-01-13", "2026-01-14", "2026-01-15"],
```

```
    "Vendedor": ["Ana", "Luis", "Ana", "Carlos", "Luis",  
                 "Ana", "Carlos", "Luis", "Ana", "Carlos",
```

```

        "Luis", "Ana", "Carlos", "Luis", "Ana"],
    "Producto": ["Laptop", "Mouse", "Teclado", "Laptop", "Monitor",
        "Mouse", "Laptop", "Teclado", "Monitor", "Mouse",
        "Laptop", "Teclado", "Monitor", "Mouse", "Laptop"],
    "Categoria": ["Tecnología"]*15,
    "Cantidad": [2,5,3,1,2,4,2,3,1,5,1,2,2,4,1],
    "PrecioUnitario": [15000,300,500,15000,4000,
        300,15000,500,4000,300,
        15000,500,4000,300,15000]
}

```

```
df = pd.DataFrame(ventas)
```

```
# Nueva columna
```

```
df["TotalVenta"] = df["Cantidad"] * df["PrecioUnitario"]
```

```
print(df)
```

```
# Total vendido por vendedor
```

```
print(df.groupby("Vendedor")["TotalVenta"].sum())
```

```
# Producto más vendido
```

```
print(df.groupby("Producto")["Cantidad"].sum().idxmax())
```

```
# Categoría con mayor ingreso
```

```
print(df.groupby("Categoria")["TotalVenta"].sum().idxmax())
```

```
# Ordenar ventas
```

```
print(df.sort_values(by="TotalVenta", ascending=False))
```

Ejercicio 18: Limpieza de datos con pandas

- Cree un DataFrame con datos incompletos de clientes.
- Incluya nombre, edad, correo, ciudad y teléfono.
- Identifique valores nulos.
- Cuente los valores nulos por columna.
- Reemplace ciudades vacías con "No especificada".
- Elimine registros que no tengan correo.
- Muestre el DataFrame limpio.

```
import pandas as pd
```

```
clientes = {  
    "Nombre": ["Ana", "Luis", "Carlos", "María", "Pedro"],  
    "Edad": [25, 30, None, 22, 35],  
    "Correo": ["ana@gmail.com", None, "carlos@gmail.com",  
               "maria@gmail.com", None],  
    "Ciudad": ["Tegucigalpa", "", "La Ceiba", "", "Comayagua"],  
    "Telefono": ["9999-1111", None, "9888-2222", "9777-3333", None]  
}
```

```
df = pd.DataFrame(clientes)
```

```
# Valores nulos
```

```
print(df.isnull())
```

```
# Contar nulos
```

```
print(df.isnull().sum())
```

```
# Reemplazar ciudades vacías
```

```
df["Ciudad"] = df["Ciudad"].replace("", "No especificada")
```

```
# Eliminar sin correo
```

```
df = df.dropna(subset=["Correo"])
```

```
# DataFrame limpio
```

```
print(df)
```

Ejercicio 19: Análisis de préstamos

- Cree un DataFrame con solicitudes de préstamo.
- Incluya cliente, monto solicitado, plazo, ingreso mensual y estado de solicitud.
- Calcule el monto promedio solicitado.
- Cuento solicitudes por estado.
- Muestre únicamente préstamos aprobados.
- Muestre clientes con monto solicitado mayor a 50,000.
- Agrupe por estado y calcule el promedio del monto solicitado.

```
import pandas as pd
```

```
prestamos = {  
    "Cliente": ["Ana", "Luis", "Carlos", "María", "Pedro",  
               "Sofía", "José", "Elena"],  
    "MontoSolicitado": [30000, 60000, 45000, 70000, 50000, 80000, 25000, 55000],  
    "Plazo": [12, 24, 18, 36, 24, 48, 12, 36],  
    "IngresoMensual": [15000, 25000, 18000, 30000, 22000, 35000, 12000, 27000],  
    "Estado": ["Aprobado", "Pendiente", "Rechazado", "Aprobado",  
              "Pendiente", "Aprobado", "Rechazado", "Aprobado"]  
}
```

```
df = pd.DataFrame(prestamos)
```

```
# Monto promedio
```

```
print("Promedio:", df["MontoSolicitado"].mean())
```

```
# Conteo por estado
```

```
print(df["Estado"].value_counts())
```

```
# Aprobados
```

```
print(df[df["Estado"] == "Aprobado"])
```

```
# Monto > 50000
```

```
print(df[df["MontoSolicitado"] > 50000])
```

```
# Agrupar por estado
```

```
print(df.groupby("Estado")["MontoSolicitado"].mean())
```

Ejercicio 20: Mini análisis de datos tipo empresa

- Cree un DataFrame con información de empleados.
- Incluya nombre, departamento, cargo, salario, años de experiencia y ciudad.
- Muestre los primeros registros.
- Muestre estadísticas generales de salarios.
- Calcule el salario promedio por departamento.
- Encuentre el empleado con salario más alto.
- Muestre empleados con más de 5 años de experiencia.
- Ordene empleados por salario.
- Cree una columna NivelExperiencia: Junior, Intermedio o Senior.

```
import pandas as pd
```

```
empleados = {
```

```
    "Nombre": ["Ana", "Luis", "Carlos", "María", "Pedro", "Sofía"],
```

```
    "Departamento": ["IT", "Ventas", "RRHH", "IT", "Finanzas", "Ventas"],
```

```
    "Cargo": ["Programador", "Vendedor", "Asistente",
```

```
              "Analista", "Contador", "Supervisor"],
```

```
    "Salario": [25000, 18000, 15000, 30000, 28000, 22000],
```

```
    "Experiencia": [2, 6, 3, 8, 10, 5],
```

```
    "Ciudad": ["Tegucigalpa", "Comayagua", "La Ceiba",
```

```
              "Siguatepeque", "Danlí", "Choluteca"]
```

```
}
```

```
df = pd.DataFrame(empleados)
```

```
# Primeros registros
```

```
print(df.head())
```

```
# Estadísticas salarios
```

```
print(df["Salario"].describe())
```

```
# Salario promedio por departamento
```

```
print(df.groupby("Departamento")["Salario"].mean())
```

```
# Salario más alto
```

```
print(df.loc[df["Salario"].idxmax()])
```

```
# Más de 5 años experiencia
print(df[df["Experiencia"] > 5])

# Ordenar por salario
print(df.sort_values(by="Salario", ascending=False))

# Nivel experiencia
def nivel(exp):
    if exp < 3:
        return "Junior"
    elif exp <= 5:
        return "Intermedio"
    else:
        return "Senior"

df["NivelExperiencia"] = df["Experiencia"].apply(nivel)
print(df)
```